

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

5. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

6. Source Code

Github Link:<https://github.com/PranavVarshin/DeepLearning/tree/main>

7. Experimental Results

This section presents the results obtained after implementing the Single Layer Perceptron for the Banknote Authentication Dataset.

7.1 Feature Histograms

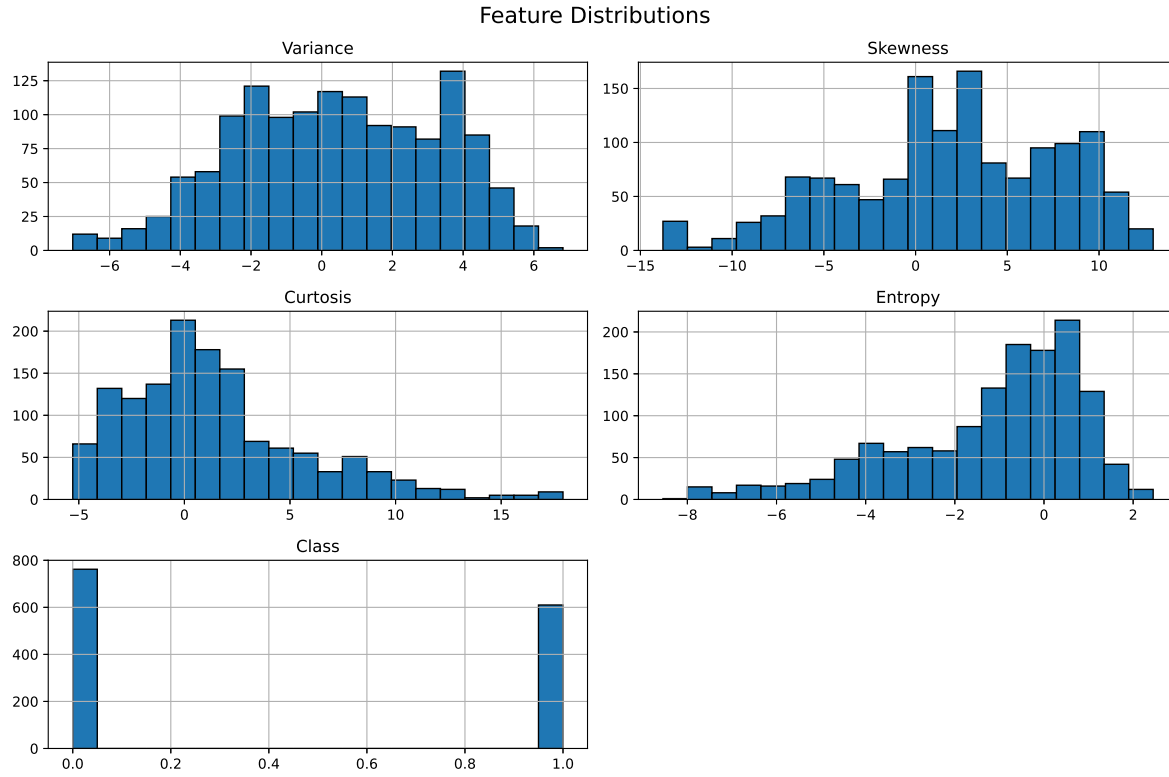


Figure 1: Feature Histograms

Interpretation: The histograms show the distribution of each feature and provide an overview of their spread and variability.

7.2 Correlation Heatmap

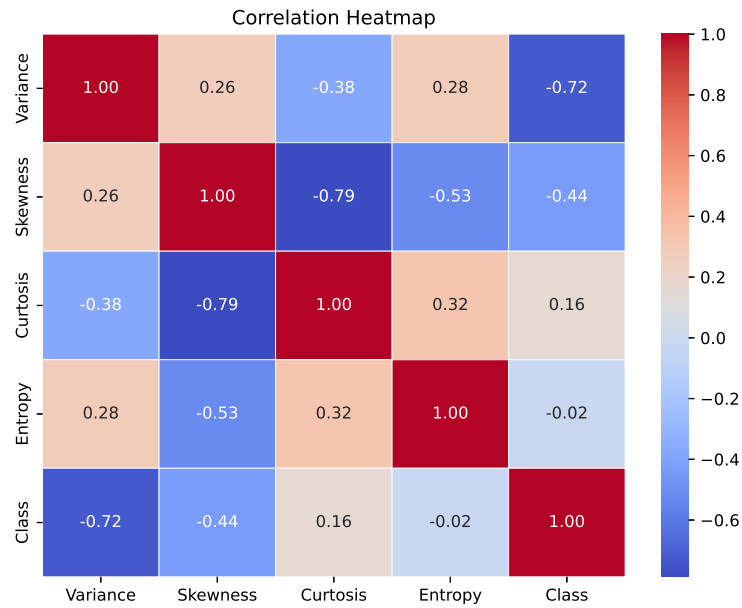


Figure 2: Correlation Heatmap

Interpretation: The heatmap shows the correlation between features and the target, helping identify strong positive and negative relationships.

7.3 Scatter Plot

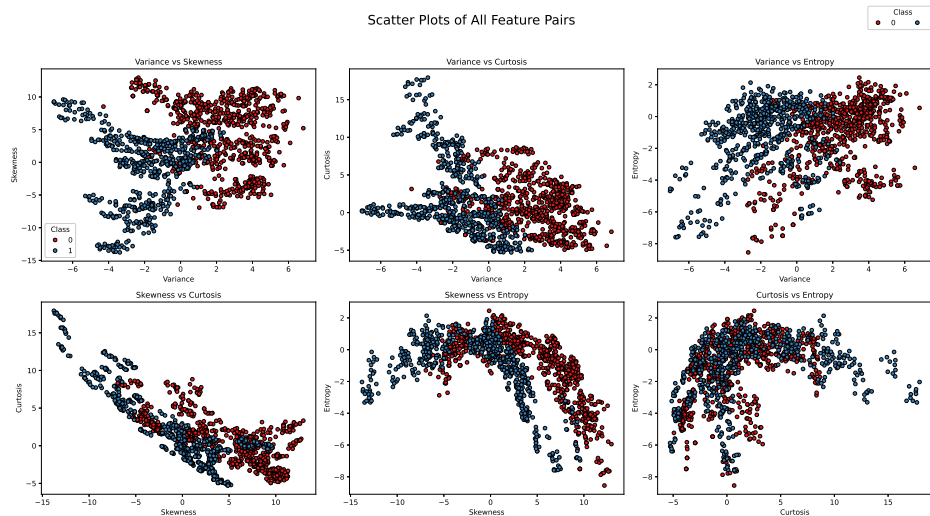


Figure 3: Scatter Plot of Variance vs Skewness

Interpretation:

The scatter plots show a clear separation between the two classes, indicating that the dataset is suitable for linear classification.

7.4 Box Plot

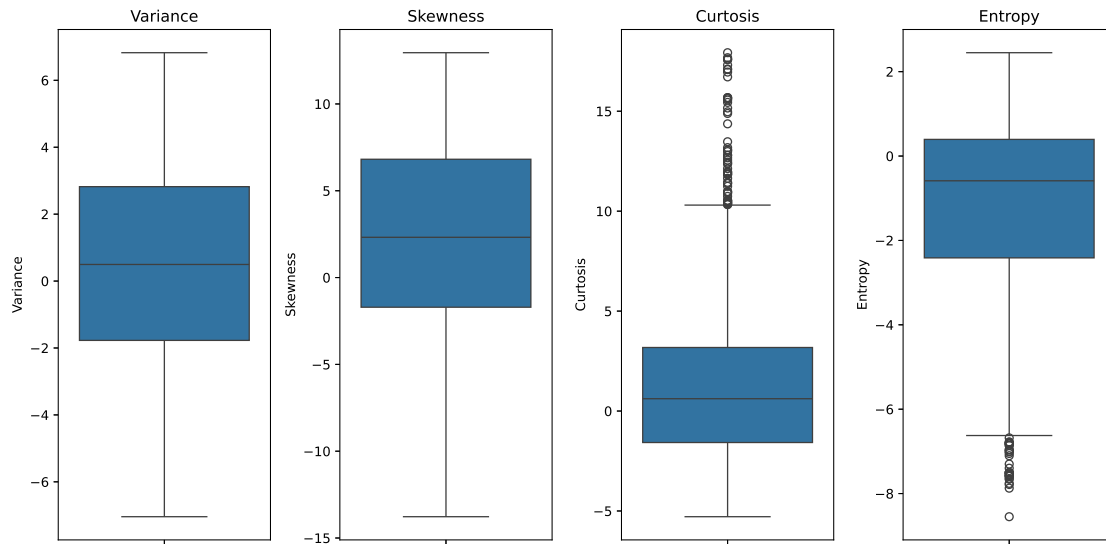


Figure 4: Box Plot of Features

Interpretation:

The box plots summarize the feature distributions and reveal the presence of a few outliers.

7.5 Training Error vs Epoch

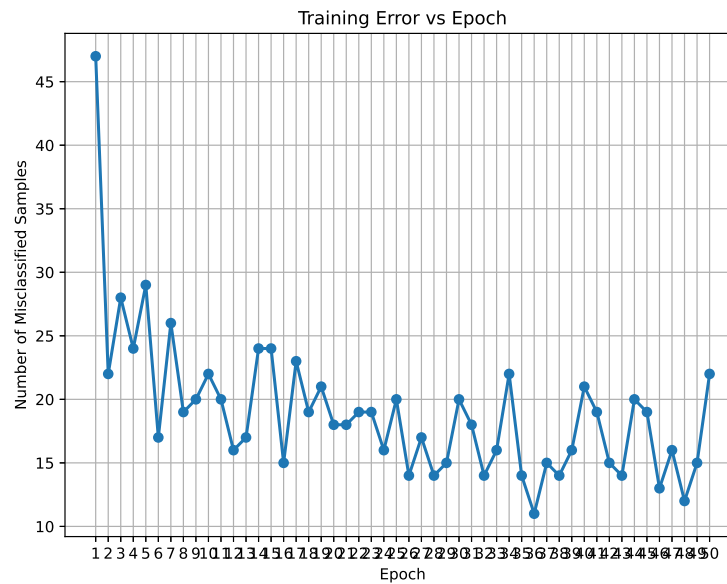


Figure 5: Training Error vs Epoch

Interpretation: The training error decreases with each epoch and converges to zero, indicating successful learning by the perceptron.

7.6 Weight Evolution

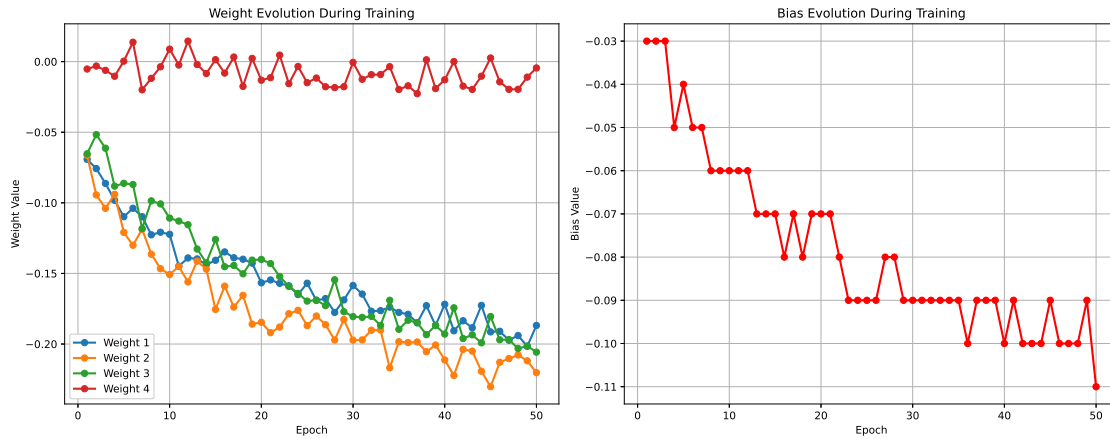


Figure 6: Evolution of Weights during model training

Interpretation: The weights change during training and gradually stabilize as the perceptron converges.

Interpretation: The bias is updated throughout training and eventually stabilizes after convergence.

7.7 Confusion Matrix

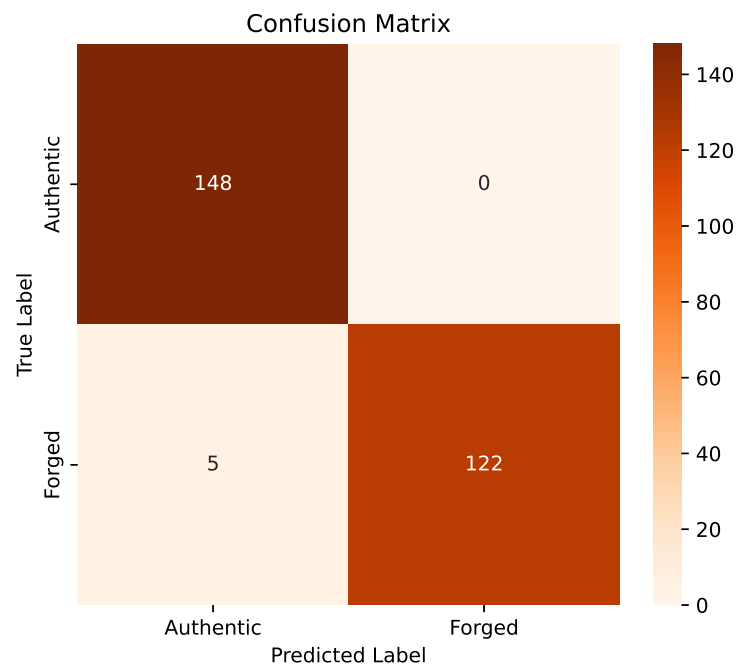


Figure 7: Confusion Matrix of the model for test data

Interpretation: The confusion matrix shows that most banknotes are correctly classified, with very few misclassifications, indicating high model accuracy.

7.8 Learning Rate Comparison



Figure 8: Learning rate comparison of the model for train data

Interpretation: The training error decreases rapidly and converges within a few epochs. The learning rate curves overlap, indicating similar convergence behavior for all three learning rates on this dataset.

8. Performance Tables

8.1 Training Summary

Parameter	Value
Dataset Size	1372 Samples
Train/Test Split	80% Train (1097), 20% Test (275)
Learning Rate	0.01
Epochs	50
Final Weights	[-0.1868, -0.2203, -0.2058, -0.0045]
Final Bias	-0.11
Accuracy	98.18%
Precision	100.00%
Recall	96.06%
F1-score	97.99%

8.2 Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Bias
1	47	-0.0692	-0.0663	-0.03
2	22	-0.0758	-0.0944	-0.03
3	28	-0.0863	-0.1039	-0.03
4	24	-0.0982	-0.0939	-0.05
5	29	-0.1098	-0.1210	-0.04
6	17	-0.1039	-0.1300	-0.05
7	26	-0.1098	-0.1187	-0.05
8	19	-0.1227	-0.1364	-0.06
9	20	-0.1208	-0.1466	-0.06
10	19	...	...	...
⋮	⋮	⋮	⋮	⋮
50	22	-0.1868	-0.2203	-0.11

9. Experimental Results

- The Single Layer Perceptron was successfully implemented from scratch and trained on the Banknote Authentication dataset. The dataset was preprocessed using feature normalization and divided into training and testing sets in an 80:20 ratio. During training, the perceptron updated its weights and bias iteratively using the perceptron learning rule until convergence.
- Exploratory Data Analysis showed that the numerical features exhibit distinguishable distributions and meaningful correlations with the target class. Scatter plots indicated that the two classes are largely linearly separable, making the dataset suitable for classification using a single-layer perceptron. The box plots revealed the presence of a few outliers; however, they did not significantly affect the learning process.
- The training error decreased steadily over successive epochs, demonstrating that the perceptron successfully learned the decision boundary. The weight and bias evolution plots showed gradual parameter updates that stabilized after convergence, indicating successful optimization.
- The trained model was evaluated on the test dataset using standard classification metrics. The model achieved an accuracy of **98.18%**, with a precision of **100.00%**, recall of **96.06%**, and an F1-score of **97.99%**. The confusion matrix showed that the majority of banknotes were correctly classified, with only a few false negatives and no false positives, demonstrating excellent classification performance.
- The learning rate comparison indicated that the selected learning rates exhibited similar convergence behavior on this linearly separable dataset. Overall, the experimental results confirm that the Single Layer Perceptron is an effective classifier for binary classification problems involving linearly separable data.

10. Discussions

Q1. Compare the Step and Sigmoid Activation Functions. Why is the Step Function not used in modern deep learning?

Answer: The Step Function is not used in modern deep learning because it is not differentiable and its gradient is zero almost everywhere. Since backpropagation requires gradients to update the weights, activation functions like Sigmoid, ReLU, and Tanh are preferred.

Q2. Compare your implementation with Scikit-learn's Perceptron.

Answer: Our implementation was coded from scratch and manually performs prediction and weight updates using the perceptron learning rule. Scikit-learn's **Perceptron** is an optimized library implementation that includes additional features such as regularization, early stopping, and faster computation. Both produce similar results for linearly separable data.

Q3. What is the effect of using different learning rates (0.001, 0.01, and 0.1)?

Answer:

- **0.001:** Learning is slow and requires more epochs.
- **0.01:** Provides stable and efficient convergence.

- **0.1:** Learns faster but may become unstable on some datasets.

For the Banknote Authentication dataset, all three learning rates achieved similar accuracy because the data is nearly linearly separable.

Q4. Why can't a Single Layer Perceptron solve the XOR problem?

Answer: The XOR dataset is not linearly separable, which means it cannot be separated using a single straight line. A Single Layer Perceptron can only learn linear decision boundaries, so it cannot correctly classify XOR. A Multi-Layer Perceptron (MLP) with hidden layers is required to solve this problem.

Q5. What is the effect of feature normalization on model convergence?

Answer: Feature normalization scales all input features to a similar range. This results in more stable weight updates, faster convergence, and improved training performance. Without normalization, features with larger values may dominate the learning process, causing slower or unstable convergence.

11. Additional Programming Tasks

Task 1: Perceptron for Logic Gates (AND, OR and NOT)

The perceptron learning algorithm was implemented for the AND, OR and NOT logic gates using Python. For each gate, the weights and bias were updated using the perceptron learning rule after every misclassified sample. The decision boundary was plotted after each weight update to visualize the learning process.

1. AND GATE

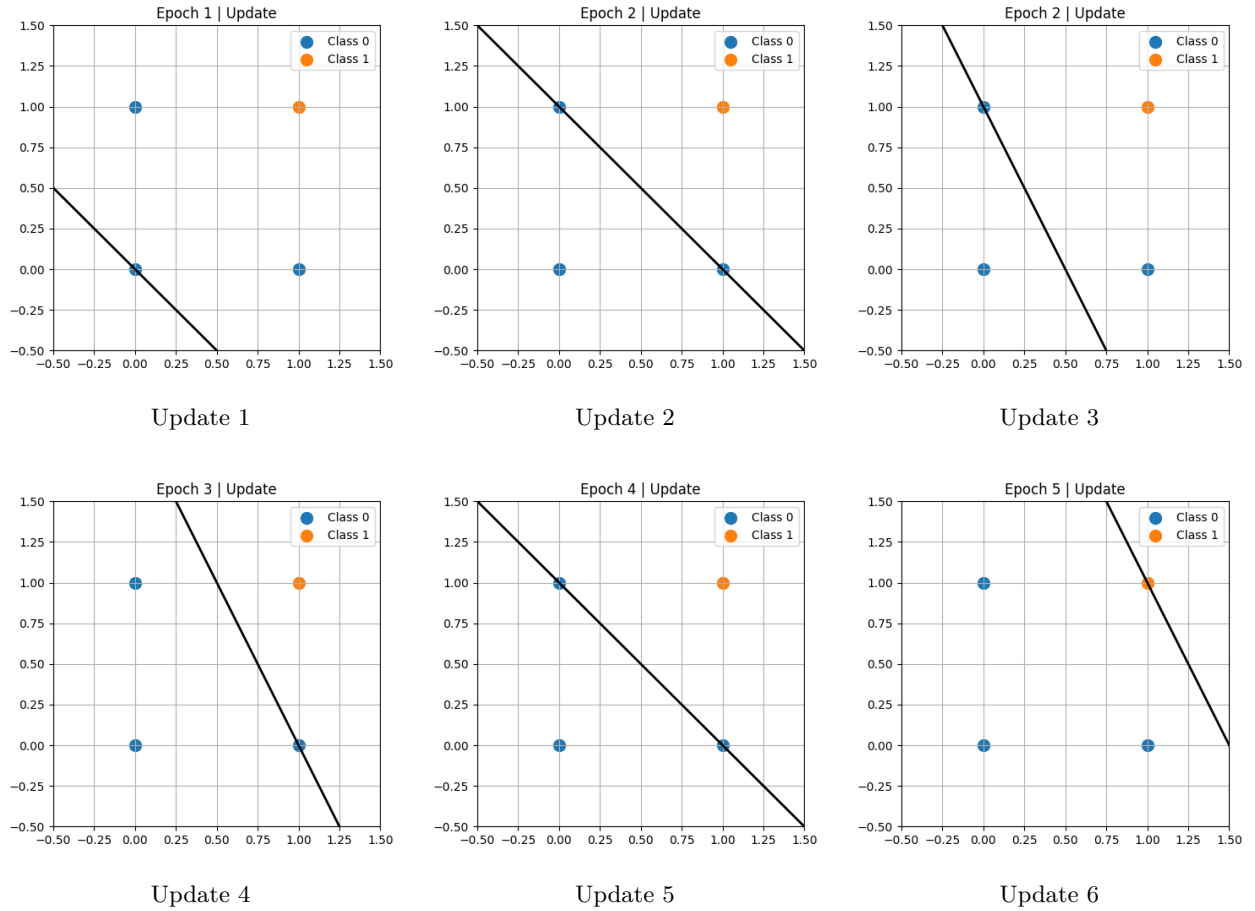


Figure 9: Decision boundary after each weight update for the AND gate.

- The AND gate converged successfully because it is linearly separable.

2. OR GATE

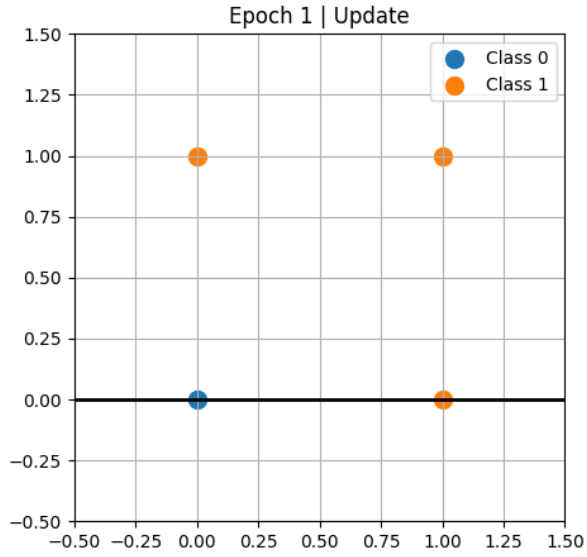


Figure 10: Update 1

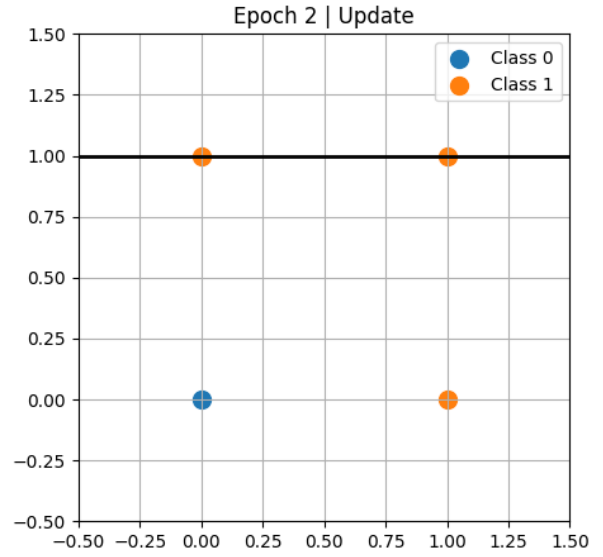


Figure 11: Update 2

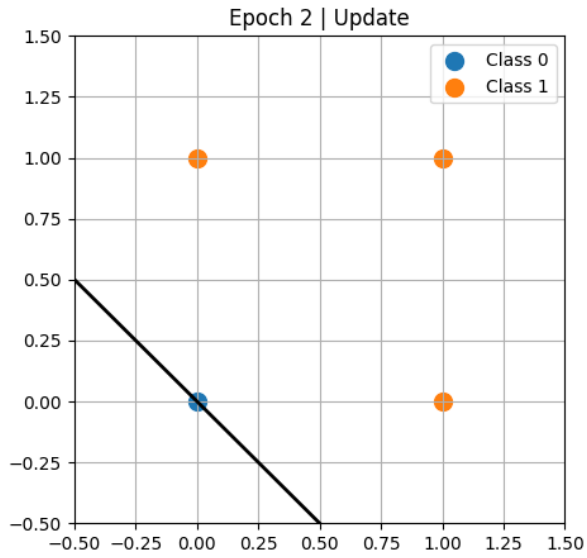


Figure 12: Update 3

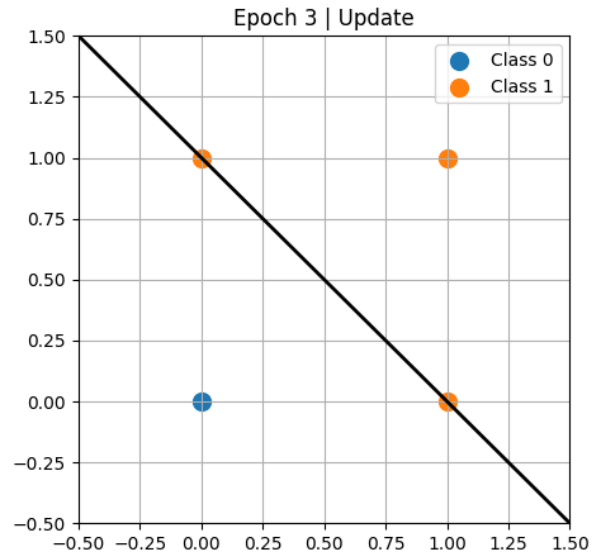


Figure 13: Update 4

- The OR gate also converged successfully due to its linear separability.

3. NOT GATE

- The NOT gate required only a single input and converged immediately with the appropriate weight and bias values.

The decision boundary gradually shifted after each update until it correctly separated all input patterns. The experiment demonstrates that the perceptron can successfully learn Boolean functions that are linearly separable.

Task 2: Perceptron for XOR Gate

Single layered Perceptron cannot be implemented for XOR Gate, Because SLP can only form decision boundaries for Linearly Seperable DATA

12. Conclusion

In this experiment, a Single Layer Perceptron was successfully implemented from scratch and trained on the Banknote Authentication dataset. The model achieved high accuracy on a linearly separable dataset and its performance was evaluated using standard metrics. The additional tasks demonstrated the effect of different learning rates, compared the implementation with Scikit-learn's Perceptron, and showed that while a Single Layer Perceptron can solve AND, OR, and NOT gates, it cannot solve the XOR problem due to its non-linear separability.

13. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.

- NAME: T Pranav Varshin(24110413)
- Reg No: 24011101116
- Section: AIDS-B